



04. PyTorch Custom Datasets Exercises Template

Welcome to the 04. PyTorch Custom Datasets exercise template.

The best way to practice PyTorch code is to write more PyTorch code.

So read the original notebook and try to complete the exercises by writing code where it's required.

Feel free to reference the original resources whenever you need but should practice writing all of the code yourself.

Resources

1. These exercises/solutions are based on [notebook 04 of the Learn PyTorch for Deep Learning course](#).
2. See a live [walkthrough of the solutions \(errors and all\) on YouTube](#).
3. See [other solutions on the course GitHub](#).

```
# Check for GPU
!nvidia-smi
```

Tue Dec 30 20:39:44 2025

NVIDIA-SMI 550.54.15				Driver Version: 550.54.15				CUDA Version: 12.4			
GPU	Name	Perf	Persistence-M	Bus-Id	Disp.A	Volatile	Uncorr. ECC				
Fan	Temp		Pwr:Usage/Cap		Memory-Usage	GPU-Util	Compute M.				
							MIG M.				
0	Tesla T4		Off	00000000:00:04.0	Off		0				
N/A	53C	P8	9W / 70W		0MiB / 15360MiB	0%	Default				N/A

Processes:								GPU Memory Usage	
GPU	GI	CI	PID	Type	Process name				
	ID	ID							
No running processes found									

```
# Import torch
import torch
from torch import nn

# Exercises require PyTorch > 1.10.0
print(torch.__version__)

# Setup device agnostic code
device = "cuda" if torch.cuda.is_available() else "cpu"
device
```

```
2.9.0+cu126
'cuda'
```

1. What are 3 methods for preventing underfitting?

The main idea of preventing underfitting is to: increase our model's predictive power.

1. **Add more layers/units to your model (increase model complexity).** This will potentially give the model more of an opportunity to learn generalizable patterns in the training data. For example, instead of using 2 hidden layers, use 4 hidden layers instead.
2. **Use transfer learning.** Transfer learning helps to prevent underfitting by leveraging already existing/working patterns from one model/dataset and using them with your own problem.
3. **Train the model for longer.** Perhaps your initial training schedule didn't give your model enough opportunity to learn patterns in the data. Training your model for longer (more epochs) may give improved results.

2. Recreate the data loading functions we built in [sections 1, 2, 3 and 4 of notebook 04](#). You should have train and test `DataLoader`'s ready to use.

```
# 1. Get data
import requests
import zipfile
from pathlib import Path

# Setup path to data folder
data_path = Path("data/")
image_path = data_path / "pizza_steak_sushi"

# If the image folder doesn't exist, download it and prepare it...
if image_path.is_dir():
    print(f"{image_path} directory exists.")
else:
    print(f"Did not find {image_path} directory, creating...")
    image_path.mkdir(parents=True, exist_ok=True)

# Download pizza, steak, sushi data (images from GitHub)
with open(data_path / "pizza_steak_sushi.zip", "wb") as f:
    request = requests.get("https://github.com/mrdbourke/pytorch-deep-learning/raw/main/data/pizza_steak_sushi.zip")
    print("Downloading pizza, steak, sushi data...")
    f.write(request.content)

# Unzip pizza, steak, sushi data
with zipfile.ZipFile(data_path/"pizza_steak_sushi.zip", "r") as zip_ref:
    print(f"Unzipping pizza, steak, sushi data to {image_path}")
    zip_ref.extractall(image_path)
```

```
Did not find data/pizza_steak_sushi directory, creating...
Downloading pizza, steak, sushi data...
Unzipping pizza, steak, sushi data to data/pizza_steak_sushi
```

```
# 2. Become one with the data
import os
def walk_through_dir(dir_path):
    """Walks through dir_path returning file counts of its contents."""
    for dirpath, dirnames, filenames in os.walk(dir_path):
        print(f"There are {len(dirnames)} directories and {len(filenames)} images in '{dirpath}'.")
```

```
walk_through_dir(image_path)
```

```
There are 2 directories and 0 images in 'data/pizza_steak_sushi'.
There are 3 directories and 0 images in 'data/pizza_steak_sushi/train'.
There are 0 directories and 75 images in 'data/pizza_steak_sushi/train/steak'.
There are 0 directories and 72 images in 'data/pizza_steak_sushi/train/sushi'.
There are 0 directories and 78 images in 'data/pizza_steak_sushi/train/pizza'.
There are 3 directories and 0 images in 'data/pizza_steak_sushi/test'.
There are 0 directories and 19 images in 'data/pizza_steak_sushi/test/steak'.
There are 0 directories and 31 images in 'data/pizza_steak_sushi/test/sushi'.
There are 0 directories and 25 images in 'data/pizza_steak_sushi/test/pizza'.
```

```
# Setup train and testing paths
train_dir = image_path / "train"
test_dir = image_path / "test"
```

```
train_dir, test_dir
```

```
(PosixPath('data/pizza_steak_sushi/train'),
 PosixPath('data/pizza_steak_sushi/test'))
```

```
# Visualize an image
import random
from PIL import Image
```

```
# Set seed
# random.seed(42)
```

```
# 1. Get all image paths (* means "any combination")
image_path_list = list(image_path.glob("*/**/*.jpg"))
print(image_path_list[:3])
```

```
# 2. Get random image path
random_image_path = random.choice(image_path_list)
print(random_image_path)

# 3. Get image class from path name
image_class = random_image_path.parent.stem
print(image_class)

# 4. Open image
img = Image.open(random_image_path)

# Print metadata
print(f"Random image path: {random_image_path}")
print(f"Image class: {image_class}")
print(f"Image height: {img.height}")
print(f"Image width: {img.width}")
img
# Image.open("/content/data/pizza_steak_sushi/test/pizza/194643.jpg")
```

```
[PosixPath('data/pizza_steak_sushi/train/steak/239025.jpg'), PosixPath('data/pizza_steak_sushi/train/steak/2881783.jp
data/pizza_steak_sushi/train/pizza/1633289.jpg
pizza
Random image path: data/pizza_steak_sushi/train/pizza/1633289.jpg
Image class: pizza
Image height: 384
Image width: 512
```



```
# Do the image visualization with matplotlib
import numpy as np
import matplotlib.pyplot as plt

# Turn the image into an array
img_as_array = np.asarray(img)

# Plot the image
plt.figure(figsize=(10, 7))
plt.imshow(img_as_array)
plt.title(f"Image class: {image_class} | Image shape: {img_as_array.shape} -> [height, width, color_channels]")
plt.axis(False);
```

Image class: pizza | Image shape: (384, 512, 3) -> [height, width, color_channels]



We've got some images in our folders.

Now we need to make them compatible with PyTorch by:

1. Transform the data into tensors.
2. Turn the tensor data into a `torch.utils.data.Dataset` and later a `torch.utils.data.DataLoader`.

```
# 3.1 Transforming data with torchvision.transforms
import torch
from torch.utils.data import DataLoader
from torchvision import datasets, transforms
```

```
# Write transform for turning images into tensors
data_transform = transforms.Compose([
    # Resize the images to 64x64x3 (64 height, 64 width, 3 color channels)
    transforms.Resize(size=(64, 64)),
    # Flip the images randomly on horizontal
    transforms.RandomHorizontalFlip(p=0.5),
    # Turn the image into a torch.Tensor
    transforms.ToTensor() # converts all pixel values from 0-255 to be between 0-1
])
```

```
random.sample(image_path_list, k=3)
```

```
[PosixPath('data/pizza_steak_sushi/test/sushi/1987407.jpg'),
 PosixPath('data/pizza_steak_sushi/train/steak/2087958.jpg'),
 PosixPath('data/pizza_steak_sushi/train/sushi/3107839.jpg')]
```

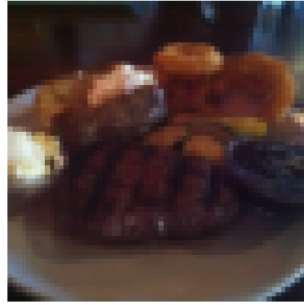
```
# Write a function to plot transformed images
def plot_transformed_images(image_paths, transform, n=3, seed=42):
    """Plots a series of random images from image_paths."""
    random.seed(seed)
    random_image_paths = random.sample(image_paths, k=n)
    for image_path in random_image_paths:
        with Image.open(image_path) as f:
            fig, ax = plt.subplots(nrows=1, ncols=2)
            ax[0].imshow(f)
            ax[0].set_title(f"Original \nsize: {f.size}")
            ax[0].axis("off")

            # Transform and plot image
            # permute() the image to make sure it's compatible with matplotlib
            transformed_image = transform(f).permute(1, 2, 0)
            ax[1].imshow(transformed_image)
            ax[1].set_title(f"Transformed \nsize: {transformed_image.shape}")
            ax[1].axis("off")

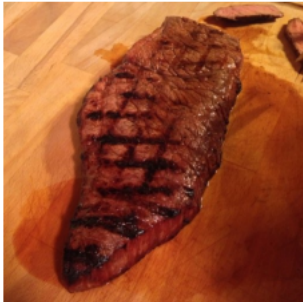
        fig.suptitle(f"Class: {image_path.parent.stem}", fontsize=16)

plot_transformed_images(image_path_list,
                        transform=data_transform,
                        n=3)
```

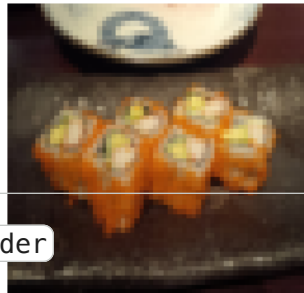
Class: steak

Original
size: (512, 512)Transformed
size: torch.Size([64, 64, 3])

Class: steak

Original
size: (512, 512)Transformed
size: torch.Size([64, 64, 3])

Class: sushi

Original
size: (512, 384)Transformed
size: torch.Size([64, 64, 3])

Load **ImageFolder**

```
# Use ImageFolder to create dataset(s)
from torchvision import datasets
train_data = datasets.ImageFolder(root=train_dir, # target folder of images
                                transform=data_transform, # transforms to perform on data (images)
                                target_transform=None) # transforms to perform on labels (if necessary)

test_data = datasets.ImageFolder(root=test_dir,
                                transform=data_transform,
                                target_transform=None)

train_data, test_data

(Dataset ImageFolder
  Number of datapoints: 225
  Root location: data/pizza_steak_sushi/train
  StandardTransform
  Transform: Compose(
    Resize(size=(64, 64), interpolation=bilinear, max_size=None, antialias=True)
    RandomHorizontalFlip(p=0.5)
    ToTensor())
```

```

    ),
    Dataset ImageFolder
    Number of datapoints: 75
    Root location: data/pizza_steak_sushi/test
    StandardTransform
    Transform: Compose(
        Resize(size=(64, 64), interpolation=bilinear, max_size=None, antialias=True)
        RandomHorizontalFlip(p=0.5)
        ToTensor()
    ))

```

```

# Get class names as a list
class_names = train_data.classes
class_names

```

```
['pizza', 'steak', 'sushi']
```

```

# Can also get class names as a dict
class_dict = train_data.class_to_idx
class_dict

```

```
{'pizza': 0, 'steak': 1, 'sushi': 2}
```

```

# Check the lengths of each dataset
len(train_data), len(test_data)

```

```
(225, 75)
```

```

# Turn train and test Datasets into DataLoaders
from torch.utils.data import DataLoader
BATCH_SIZE = 1
train_dataloader = DataLoader(dataset=train_data,
                              batch_size=BATCH_SIZE,
                              num_workers=os.cpu_count(),
                              shuffle=True)

test_dataloader = DataLoader(dataset=test_data,
                             batch_size=BATCH_SIZE,
                             num_workers=os.cpu_count(),
                             shuffle=False)

train_dataloader, test_dataloader

```

```

(<torch.utils.data.dataloader.DataLoader at 0x7b541f71fef0>,
 <torch.utils.data.dataloader.DataLoader at 0x7b541f71da00>)

```

```

# How many batches of images are in our data loaders?
len(train_dataloader), len(test_dataloader)

```

```
(225, 75)
```

```
img, label = next(iter(train_dataloader))
```

```

# Batch size will now be 1, try changing the batch_size parameter above and see what happens
print(f"Image shape: {img.shape} -> [batch_size, color_channels, height, width]")
print(f"Label shape: {label.shape}")

```

```

Image shape: torch.Size([1, 3, 64, 64]) -> [batch_size, color_channels, height, width]
Label shape: torch.Size([1])

```

3. Recreate `model_0` we built in section 7 of notebook 04.

```

import torch
from torch import nn
class TinyVGG(nn.Module):
    def __init__(self, input_shape, hidden_units, output_shape):
        super().__init__()
        self.conv_block_1 = nn.Sequential(
            nn.Conv2d(in_channels=input_shape,
                      out_channels=hidden_units,
                      kernel_size=3,
                      stride=1,
                      padding=1
            ),

```

```

        nn.ReLU(),
        nn.Conv2d(in_channels=hidden_units,
                  out_channels=hidden_units,
                  kernel_size=3,
                  stride=1,
                  padding=1
                ),
        nn.ReLU(),
        nn.MaxPool2d(kernel_size=2)
    )
    self.conv_block_2 = nn.Sequential(
        nn.Conv2d(in_channels=hidden_units,
                  out_channels=hidden_units,
                  kernel_size=3,
                  stride=1,
                  padding=1
                ),
        nn.ReLU(),
        nn.Conv2d(in_channels=hidden_units,
                  out_channels=hidden_units,
                  kernel_size=3,
                  stride=1,
                  padding=1
                ),
        nn.ReLU(),
        nn.MaxPool2d(kernel_size=2)
    )
    self.classifier = nn.Sequential(nn.Flatten(),
                                    nn.Linear(in_features=hidden_units*16*16,
                                              out_features=output_shape))

def forward(self,x):
    x = self.conv_block_1(x)
    x = self.conv_block_2(x)
    x = self.classifier(x)
    return x

```

```

model_0 = TinyVGG(input_shape = 3,
                  hidden_units=10,
                  output_shape=len(class_names)).to(device)

model_0

```

```

TinyVGG(
  (conv_block_1): Sequential(
    (0): Conv2d(3, 10, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (1): ReLU()
    (2): Conv2d(10, 10, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (3): ReLU()
    (4): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
  )
  (conv_block_2): Sequential(
    (0): Conv2d(10, 10, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (1): ReLU()
    (2): Conv2d(10, 10, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (3): ReLU()
    (4): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
  )
  (classifier): Sequential(
    (0): Flatten(start_dim=1, end_dim=-1)
    (1): Linear(in_features=2560, out_features=3, bias=True)
  )
)

```

```
len(class_names)
```

```
3
```

```
16*16*10
```

```
2560
```

```

# Pass dummy data through model
dummy_x = torch.rand(size=[1, 3, 64, 64])
model_0(dummy_x.to(device))

```

```
tensor([[0.0503, 0.0019, 0.0235]], device='cuda:0', grad_fn=<AddmmBackward0>)
```


4. Create training and testing functions for `model_0`.

```
def train_step(model: torch.nn.Module,
               dataloader: torch.utils.data.DataLoader,
               loss_fn: torch.nn.Module,
               optimizer: torch.optim.Optimizer):

    # Put the model in train mode
    model.train()

    # Setup train loss and train accuracy values
    train_loss, train_acc = 0, 0

    # Loop through data loader and data batches
    for batch, (X, y) in enumerate(dataloader):
        # Send data to target device
        X, y = X.to(device), y.to(device)

        # 1. Forward pass
        y_pred = model(X)

        # 2. Calculate and accumulate loss
        loss = loss_fn(y_pred, y)
        train_loss += loss.item()

        # 3. Optimizer zero grad
        optimizer.zero_grad()

        # 4. Loss backward
        loss.backward()

        # 5. Optimizer step
        optimizer.step()

    # Calculate and accumulate accuracy metric across all batches
    y_pred_class = torch.argmax(torch.softmax(y_pred, dim=1), dim=1)
    train_acc += (y_pred_class == y).sum().item() / len(y_pred)

    # Adjust metrics to get average loss and average accuracy per batch
    train_loss = train_loss / len(dataloader)
    train_acc = train_acc / len(dataloader)
    return train_loss, train_acc
```

```
def test_step(model: torch.nn.Module,
              dataloader: torch.utils.data.DataLoader,
              loss_fn: torch.nn.Module):

    # Put model in eval mode
    model.eval()

    # Setup the test loss and test accuracy values
    test_loss, test_acc = 0, 0

    # Turn on inference context manager
    with torch.inference_mode():
        # Loop through DataLoader batches

        for batch, (X, y) in enumerate(dataloader):
            # Send data to target device
            X, y = X.to(device), y.to(device)

            # 1. Forward pass - get logits
            test_pred_logits = model(X)

            # 2. Calculate and accumulate loss
            loss = loss_fn(test_pred_logits, y)
            test_loss += loss.item()

            # Calculate and accumulate accuracy
            test_pred_labels = test_pred_logits.argmax(dim=1)
            test_acc += ((test_pred_labels == y).sum().item() / len(test_pred_labels))
```



```
# Adjust metrics to get average loss and accuracy per batch
test_loss = test_loss / len(dataloader)
test_acc = test_acc / len(dataloader)
return test_loss, test_acc
```

```
from tqdm.auto import tqdm

def train(model: torch.nn.Module,
          train_dataloader: torch.utils.data.DataLoader,
          test_dataloader: torch.utils.data.DataLoader,
          optimizer: torch.optim.Optimizer,
          loss_fn: torch.nn.Module = nn.CrossEntropyLoss(),
          epochs: int = 5):

    # Create results dictionary
    results = {"train_loss": [],
               "train_acc": [],
               "test_loss": [],
               "test_acc": []}

    # Loop through the training and testing steps for a number of epochs
    for epoch in tqdm(range(epochs)):
        # Train step
        train_loss, train_acc = train_step(model=model,
                                           dataloader=train_dataloader,
                                           loss_fn=loss_fn,
                                           optimizer=optimizer)

        # Test step
        test_loss, test_acc = test_step(model=model,
                                         dataloader=test_dataloader,
                                         loss_fn=loss_fn)

        # Print out what's happening
        print(f"Epoch: {epoch+1} | "
              f"train_loss: {train_loss:.4f} | "
              f"train_acc: {train_acc:.4f} | "
              f"test_loss: {test_loss:.4f} | "
              f"test_acc: {test_acc:.4f}")

        # Update the results dictionary
        results["train_loss"].append(train_loss)
        results["train_acc"].append(train_acc)
        results["test_loss"].append(test_loss)
        results["test_acc"].append(test_acc)

    # Return the results dictionary
    return results
```

5. Try training the model you made in exercise 3 for 5, 20 and 50 epochs, what happens to the results?

- Use `torch.optim.Adam()` with a learning rate of 0.001 as the optimizer.

```
# Train for 5 epochs
torch.manual_seed(42)
torch.cuda.manual_seed(42)
model_0 = TinyVGG(input_shape=3,
                  hidden_units=10,
                  output_shape=len(class_names)).to(device)

loss_fn=nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model_0.parameters(), lr=0.001)

model_0_results = train(model=model_0,
                        train_dataloader=train_dataloader,
                        test_dataloader=test_dataloader,
                        optimizer=optimizer,
                        epochs=5)
```

```

100%
5/5 [00:08<00:00, 1.63s/it]
Epoch: 1 | train_loss: 1.1182 | train_acc: 0.2889 | test_loss: 1.0962 | test_acc: 0.3200
Epoch: 2 | train_loss: 1.1025 | train_acc: 0.3467 | test_loss: 1.1054 | test_acc: 0.2533
Epoch: 3 | train_loss: 1.1006 | train_acc: 0.3067 | test_loss: 1.0999 | test_acc: 0.3333
Epoch: 4 | train_loss: 1.0994 | train_acc: 0.3422 | test_loss: 1.1072 | test_acc: 0.3333
Epoch: 5 | train_loss: 1.0999 | train_acc: 0.2889 | test_loss: 1.0995 | test_acc: 0.3333

```

```

# Train for 20 epochs
torch.manual_seed(42)
torch.cuda.manual_seed(42)
model_1 = TinyVGG(input_shape=3,
                  hidden_units=10,
                  output_shape=len(class_names)).to(device)

loss_fn = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model_1.parameters(), lr=0.001)

model_1_results = train(model=model_1,
                       train_dataloader=train_dataloader,
                       test_dataloader=test_dataloader,
                       optimizer=optimizer,
                       epochs=20)

```

```

100%
20/20 [00:33<00:00, 1.67s/it]
Epoch: 1 | train_loss: 1.1182 | train_acc: 0.2889 | test_loss: 1.0962 | test_acc: 0.3200
Epoch: 2 | train_loss: 1.1025 | train_acc: 0.3511 | test_loss: 1.1062 | test_acc: 0.2533
Epoch: 3 | train_loss: 1.1005 | train_acc: 0.3422 | test_loss: 1.0995 | test_acc: 0.3333
Epoch: 4 | train_loss: 1.0990 | train_acc: 0.3289 | test_loss: 1.0998 | test_acc: 0.3333
Epoch: 5 | train_loss: 1.0991 | train_acc: 0.3422 | test_loss: 1.1001 | test_acc: 0.3333
Epoch: 6 | train_loss: 1.0993 | train_acc: 0.3511 | test_loss: 1.1006 | test_acc: 0.3333
Epoch: 7 | train_loss: 1.0984 | train_acc: 0.3467 | test_loss: 1.1010 | test_acc: 0.3333
Epoch: 8 | train_loss: 1.1021 | train_acc: 0.3289 | test_loss: 1.1043 | test_acc: 0.3333
Epoch: 9 | train_loss: 1.0997 | train_acc: 0.3467 | test_loss: 1.1012 | test_acc: 0.3333
Epoch: 10 | train_loss: 1.0987 | train_acc: 0.3467 | test_loss: 1.1015 | test_acc: 0.3333
Epoch: 11 | train_loss: 1.1237 | train_acc: 0.3556 | test_loss: 1.0866 | test_acc: 0.4667
Epoch: 12 | train_loss: 1.0968 | train_acc: 0.3956 | test_loss: 1.1129 | test_acc: 0.2933
Epoch: 13 | train_loss: 1.0669 | train_acc: 0.4089 | test_loss: 1.0903 | test_acc: 0.3867
Epoch: 14 | train_loss: 1.0014 | train_acc: 0.4933 | test_loss: 1.0305 | test_acc: 0.4000
Epoch: 15 | train_loss: 0.9828 | train_acc: 0.5156 | test_loss: 1.0772 | test_acc: 0.3733
Epoch: 16 | train_loss: 0.9532 | train_acc: 0.5244 | test_loss: 1.1954 | test_acc: 0.3733
Epoch: 17 | train_loss: 0.9425 | train_acc: 0.5511 | test_loss: 1.1270 | test_acc: 0.3467
Epoch: 18 | train_loss: 0.9407 | train_acc: 0.5867 | test_loss: 1.1608 | test_acc: 0.3333
Epoch: 19 | train_loss: 0.9173 | train_acc: 0.5511 | test_loss: 1.1410 | test_acc: 0.3200
Epoch: 20 | train_loss: 0.9008 | train_acc: 0.5511 | test_loss: 1.1847 | test_acc: 0.3067

```

```

# Train for 50 epochs
torch.manual_seed(42)
torch.cuda.manual_seed(42)
model_2 = TinyVGG(input_shape=3,
                  hidden_units=10,
                  output_shape=len(class_names)).to(device)

loss_fn = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model_2.parameters(), lr=0.001)

model_2_results = train(model=model_2,
                       train_dataloader=train_dataloader,
                       test_dataloader=test_dataloader,
                       optimizer=optimizer,
                       epochs=50)

```

```

100%
50/50 [01:23<00:00, 1.62s/it]

Epoch: 1 | train_loss: 1.1180 | train_acc: 0.2844 | test_loss: 1.0964 | test_acc: 0.3333
Epoch: 2 | train_loss: 1.1012 | train_acc: 0.3244 | test_loss: 1.0998 | test_acc: 0.2533
Epoch: 3 | train_loss: 1.0993 | train_acc: 0.2933 | test_loss: 1.0999 | test_acc: 0.3333
Epoch: 4 | train_loss: 1.0990 | train_acc: 0.3289 | test_loss: 1.1002 | test_acc: 0.3867
Epoch: 5 | train_loss: 1.0989 | train_acc: 0.3467 | test_loss: 1.1007 | test_acc: 0.3333
Epoch: 6 | train_loss: 1.0991 | train_acc: 0.3467 | test_loss: 1.1010 | test_acc: 0.3333
Epoch: 7 | train_loss: 1.0986 | train_acc: 0.3467 | test_loss: 1.1013 | test_acc: 0.3333
Epoch: 8 | train_loss: 1.0983 | train_acc: 0.3422 | test_loss: 1.1013 | test_acc: 0.3333
Epoch: 9 | train_loss: 1.0993 | train_acc: 0.3467 | test_loss: 1.0998 | test_acc: 0.3333
Epoch: 10 | train_loss: 1.1773 | train_acc: 0.3511 | test_loss: 1.1001 | test_acc: 0.3333
Epoch: 11 | train_loss: 1.0991 | train_acc: 0.3467 | test_loss: 1.1017 | test_acc: 0.3333
Epoch: 12 | train_loss: 1.0996 | train_acc: 0.3378 | test_loss: 1.1011 | test_acc: 0.3333
Epoch: 13 | train_loss: 1.0965 | train_acc: 0.3600 | test_loss: 1.1093 | test_acc: 0.2933
Epoch: 14 | train_loss: 1.0735 | train_acc: 0.4222 | test_loss: 1.2542 | test_acc: 0.3867
Epoch: 15 | train_loss: 1.0548 | train_acc: 0.5200 | test_loss: 1.0813 | test_acc: 0.4000
Epoch: 16 | train_loss: 0.9564 | train_acc: 0.5778 | test_loss: 1.0972 | test_acc: 0.4400
Epoch: 17 | train_loss: 0.8945 | train_acc: 0.6356 | test_loss: 1.0881 | test_acc: 0.4800
Epoch: 18 | train_loss: 0.8741 | train_acc: 0.6267 | test_loss: 1.0686 | test_acc: 0.3867
Epoch: 19 | train_loss: 0.8625 | train_acc: 0.6133 | test_loss: 1.0401 | test_acc: 0.4400
Epoch: 20 | train_loss: 0.7785 | train_acc: 0.6667 | test_loss: 1.1010 | test_acc: 0.4667
Epoch: 21 | train_loss: 0.7644 | train_acc: 0.6933 | test_loss: 1.0704 | test_acc: 0.4667
Epoch: 22 | train_loss: 0.7150 | train_acc: 0.7022 | test_loss: 1.1541 | test_acc: 0.4800
Epoch: 23 | train_loss: 0.6982 | train_acc: 0.6933 | test_loss: 1.1523 | test_acc: 0.4800
Epoch: 24 | train_loss: 0.6588 | train_acc: 0.7467 | test_loss: 1.2747 | test_acc: 0.4533
Epoch: 25 | train_loss: 0.6298 | train_acc: 0.7156 | test_loss: 1.1604 | test_acc: 0.4267
Epoch: 26 | train_loss: 0.5670 | train_acc: 0.7689 | test_loss: 1.1971 | test_acc: 0.4667
Epoch: 27 | train_loss: 0.5903 | train_acc: 0.7511 | test_loss: 1.3428 | test_acc: 0.4933
Epoch: 28 | train_loss: 0.5697 | train_acc: 0.7600 | test_loss: 1.5729 | test_acc: 0.4267
Epoch: 29 | train_loss: 0.5100 | train_acc: 0.8000 | test_loss: 1.3758 | test_acc: 0.4533
Epoch: 30 | train_loss: 0.4684 | train_acc: 0.8044 | test_loss: 1.5816 | test_acc: 0.4533
Epoch: 31 | train_loss: 0.4291 | train_acc: 0.8311 | test_loss: 1.6313 | test_acc: 0.5067
Epoch: 32 | train_loss: 0.3950 | train_acc: 0.8667 | test_loss: 1.9707 | test_acc: 0.4267
Epoch: 33 | train_loss: 0.3487 | train_acc: 0.8844 | test_loss: 1.7817 | test_acc: 0.4400
Epoch: 34 | train_loss: 0.3216 | train_acc: 0.8800 | test_loss: 1.6520 | test_acc: 0.5600
Epoch: 35 | train_loss: 0.2798 | train_acc: 0.8933 | test_loss: 2.2177 | test_acc: 0.5467
Epoch: 36 | train_loss: 0.2431 | train_acc: 0.9200 | test_loss: 2.4461 | test_acc: 0.4400
Epoch: 37 | train_loss: 0.2142 | train_acc: 0.9289 | test_loss: 2.6501 | test_acc: 0.4400
Epoch: 38 | train_loss: 0.2089 | train_acc: 0.9067 | test_loss: 2.2774 | test_acc: 0.4533
Epoch: 39 | train_loss: 0.1225 | train_acc: 0.9733 | test_loss: 2.6509 | test_acc: 0.4933
Epoch: 40 | train_loss: 0.0968 | train_acc: 0.9733 | test_loss: 3.4175 | test_acc: 0.4667
Epoch: 41 | train_loss: 0.0917 | train_acc: 0.9778 | test_loss: 3.1553 | test_acc: 0.4400
Epoch: 42 | train_loss: 0.1462 | train_acc: 0.9378 | test_loss: 3.4279 | test_acc: 0.4400
Epoch: 43 | train_loss: 0.0552 | train_acc: 0.9911 | test_loss: 3.7901 | test_acc: 0.4133
Epoch: 44 | train_loss: 0.0363 | train_acc: 0.9911 | test_loss: 3.7063 | test_acc: 0.4800
Epoch: 45 | train_loss: 0.1737 | train_acc: 0.9511 | test_loss: 3.8213 | test_acc: 0.4133
Epoch: 46 | train_loss: 0.0555 | train_acc: 0.9867 | test_loss: 4.2747 | test_acc: 0.3733
Epoch: 47 | train_loss: 0.0594 | train_acc: 0.9867 | test_loss: 4.7068 | test_acc: 0.3600
Epoch: 48 | train_loss: 0.0312 | train_acc: 0.9911 | test_loss: 4.9402 | test_acc: 0.4000
Epoch: 49 | train_loss: 0.0590 | train_acc: 0.9822 | test_loss: 3.7667 | test_acc: 0.4933
Epoch: 50 | train_loss: 0.0958 | train_acc: 0.9600 | test_loss: 3.4388 | test_acc: 0.4533

```

It looks like our model is starting to overfit towards the end (performing far better on the training data than on the testing data).

In order to fix this, we'd have to introduce ways of preventing overfitting.

6. Double the number of hidden units in your model and train it for 20 epochs, what happens to the results?

```

# Double the number of hidden units and train for 20 epochs
torch.manual_seed(42)
torch.cuda.manual_seed(42)
model_3 = TinyVGG(input_shape=3,
                   hidden_units=20, # use 20 hidden units instead of 10
                   output_shape=len(class_names)).to(device)

loss_fn = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model_3.parameters(), lr=0.001)

model_3_results = train(model=model_3,
                        train_dataloader=train_dataloader,
                        test_dataloader=test_dataloader,
                        optimizer=optimizer,
                        epochs=20) # train for 20 epochs

```

```

100%
20/20 [00:32<00:00, 1.57s/it]
Epoch: 1 | train_loss: 1.0990 | train_acc: 0.3689 | test_loss: 1.0796 | test_acc: 0.3333
Epoch: 2 | train_loss: 1.0316 | train_acc: 0.4444 | test_loss: 1.0238 | test_acc: 0.4667
Epoch: 3 | train_loss: 0.9523 | train_acc: 0.5511 | test_loss: 0.9911 | test_acc: 0.4533
Epoch: 4 | train_loss: 0.8726 | train_acc: 0.6133 | test_loss: 0.9804 | test_acc: 0.4800
Epoch: 5 | train_loss: 0.8649 | train_acc: 0.6000 | test_loss: 1.0376 | test_acc: 0.4533
Epoch: 6 | train_loss: 0.8502 | train_acc: 0.6444 | test_loss: 0.9741 | test_acc: 0.4800
Epoch: 7 | train_loss: 0.8077 | train_acc: 0.6756 | test_loss: 1.0683 | test_acc: 0.5200
Epoch: 8 | train_loss: 0.7186 | train_acc: 0.6933 | test_loss: 1.0681 | test_acc: 0.4933
Epoch: 9 | train_loss: 0.6853 | train_acc: 0.7111 | test_loss: 1.0501 | test_acc: 0.4533
Epoch: 10 | train_loss: 0.6290 | train_acc: 0.7333 | test_loss: 1.1587 | test_acc: 0.5200
Epoch: 11 | train_loss: 0.6096 | train_acc: 0.7467 | test_loss: 1.1036 | test_acc: 0.5200
Epoch: 12 | train_loss: 0.5318 | train_acc: 0.7911 | test_loss: 1.2360 | test_acc: 0.5600
Epoch: 13 | train_loss: 0.4252 | train_acc: 0.8133 | test_loss: 1.7130 | test_acc: 0.5467
Epoch: 14 | train_loss: 0.4379 | train_acc: 0.8444 | test_loss: 1.6944 | test_acc: 0.5467
Epoch: 15 | train_loss: 0.3665 | train_acc: 0.8533 | test_loss: 1.4481 | test_acc: 0.5467
Epoch: 16 | train_loss: 0.2807 | train_acc: 0.8889 | test_loss: 1.8729 | test_acc: 0.5733
Epoch: 17 | train_loss: 0.2250 | train_acc: 0.9067 | test_loss: 2.2892 | test_acc: 0.4800
Epoch: 18 | train_loss: 0.2305 | train_acc: 0.9333 | test_loss: 2.1959 | test_acc: 0.5867
Epoch: 19 | train_loss: 0.2267 | train_acc: 0.9333 | test_loss: 2.1559 | test_acc: 0.4933
Epoch: 20 | train_loss: 0.2123 | train_acc: 0.9333 | test_loss: 2.1165 | test_acc: 0.4667

```

It looks like the model is still overfitting, even when changing the number of hidden units.

To fix this, we'd have to look at ways to prevent overfitting with our model.

7. Double the data you're using with your model from step 6 and train it for 20 epochs, what happens to the results?

- **Note:** You can use the [custom data creation notebook](#) to scale up your Food101 dataset.
- You can also find the [already formatted double data \(20% instead of 10% subset\) dataset on GitHub](#), you will need to write download code like in exercise 2 to get it into this notebook.

```

# Download 20% data for Pizza/Steak/Sushi from GitHub
import requests
import zipfile
from pathlib import Path

# Setup path to data folder
data_path = Path("data/")
image_path = data_path / "pizza_steak_sushi_20_percent"

# If the image folder doesn't exist, download it and prepare it...
if image_path.is_dir():
    print(f"{image_path} directory exists.")
else:
    print(f"Did not find {image_path} directory, creating one...")
    image_path.mkdir(parents=True, exist_ok=True)

# Download pizza, steak, sushi data
with open(data_path / "pizza_steak_sushi_20_percent.zip", "wb") as f:
    request = requests.get("https://github.com/mrdbourke/pytorch-deep-learning/raw/main/data/pizza_steak_sushi_20_percent.zip")
    print("Downloading pizza, steak, sushi 20% data...")
    f.write(request.content)

# Unzip pizza, steak, sushi data
with zipfile.ZipFile(data_path / "pizza_steak_sushi_20_percent.zip", "r") as zip_ref:
    print("Unzipping pizza, steak, sushi 20% data...")
    zip_ref.extractall(image_path)

Did not find data/pizza_steak_sushi_20_percent directory, creating one...
Downloading pizza, steak, sushi 20% data...
Unzipping pizza, steak, sushi 20% data...

```

```

# See how many images we have
walk_through_dir(image_path)

```

```

There are 2 directories and 0 images in 'data/pizza_steak_sushi_20_percent'.
There are 3 directories and 0 images in 'data/pizza_steak_sushi_20_percent/train'.
There are 0 directories and 146 images in 'data/pizza_steak_sushi_20_percent/train/steak'.
There are 0 directories and 150 images in 'data/pizza_steak_sushi_20_percent/train/sushi'.
There are 0 directories and 154 images in 'data/pizza_steak_sushi_20_percent/train/pizza'.

```

```

There are 3 directories and 0 images in 'data/pizza_steak_sushi_20_percent/test'.
There are 0 directories and 58 images in 'data/pizza_steak_sushi_20_percent/test/steak'.
There are 0 directories and 46 images in 'data/pizza_steak_sushi_20_percent/test/sushi'.
There are 0 directories and 46 images in 'data/pizza_steak_sushi_20_percent/test/pizza'.

```

Excellent, we now have double the training and testing images...

```

# Create the train and test paths
train_data_20_percent_path = image_path / "train"
test_data_20_percent_path = image_path / "test"

train_data_20_percent_path, test_data_20_percent_path

(PosixPath('data/pizza_steak_sushi_20_percent/train'),
 PosixPath('data/pizza_steak_sushi_20_percent/test'))

```

```

from torchvision.datasets import ImageFolder
from torchvision import transforms
from torch.utils.data import DataLoader

simple_transform = transforms.Compose([
    transforms.Resize((64, 64)),
    transforms.ToTensor()
])

train_data_20_percent = ImageFolder(train_data_20_percent_path,
                                     transform=simple_transform)

test_data_20_percent = ImageFolder(test_data_20_percent_path,
                                    transform=simple_transform)

# Create dataloaders
train_dataloader_20_percent = DataLoader(train_data_20_percent,
                                         batch_size=32,
                                         num_workers=os.cpu_count(),
                                         shuffle=True)

test_dataloader_20_percent = DataLoader(test_data_20_percent,
                                         batch_size=32,
                                         num_workers=os.cpu_count(),
                                         shuffle=False)

```

```

# Train a model with increased amount of data
torch.manual_seed(42)
torch.cuda.manual_seed(42)
model_4 = TinyVGG(input_shape=3,
                  hidden_units=20, # use 20 hidden units instead of 10
                  output_shape=len(class_names)).to(device)

loss_fn = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model_4.parameters(), lr=0.001)

model_4_results = train(model=model_4,
                        train_dataloader=train_dataloader_20_percent, # use double the training data
                        test_dataloader=test_dataloader_20_percent, # use double the testing data
                        optimizer=optimizer,
                        epochs=20) # train for 20 epochs

```

```

100%
20/20 [00:36<00:00, 1.74s/it]
Epoch: 1 | train_loss: 1.1042 | train_acc: 0.3312 | test_loss: 1.1004 | test_acc: 0.2875
Epoch: 2 | train_loss: 1.0833 | train_acc: 0.3917 | test_loss: 1.0684 | test_acc: 0.4136
Epoch: 3 | train_loss: 1.0271 | train_acc: 0.4500 | test_loss: 0.9465 | test_acc: 0.5722
Epoch: 4 | train_loss: 0.9218 | train_acc: 0.5500 | test_loss: 0.9140 | test_acc: 0.5909
Epoch: 5 | train_loss: 0.8626 | train_acc: 0.6271 | test_loss: 0.9144 | test_acc: 0.5273
Epoch: 6 | train_loss: 0.8260 | train_acc: 0.6562 | test_loss: 1.0457 | test_acc: 0.4807
Epoch: 7 | train_loss: 0.8979 | train_acc: 0.5521 | test_loss: 0.8933 | test_acc: 0.5341
Epoch: 8 | train_loss: 0.7703 | train_acc: 0.6958 | test_loss: 0.8877 | test_acc: 0.6136
Epoch: 9 | train_loss: 0.7951 | train_acc: 0.6458 | test_loss: 0.8980 | test_acc: 0.5750
Epoch: 10 | train_loss: 0.7735 | train_acc: 0.6833 | test_loss: 0.9088 | test_acc: 0.5915
Epoch: 11 | train_loss: 0.7393 | train_acc: 0.6667 | test_loss: 0.8598 | test_acc: 0.6568
Epoch: 12 | train_loss: 0.6543 | train_acc: 0.7354 | test_loss: 0.9358 | test_acc: 0.5852
Epoch: 13 | train_loss: 0.6101 | train_acc: 0.7500 | test_loss: 1.0142 | test_acc: 0.5244
Epoch: 14 | train_loss: 0.5812 | train_acc: 0.7625 | test_loss: 1.0818 | test_acc: 0.5420
Epoch: 15 | train_loss: 0.5743 | train_acc: 0.7771 | test_loss: 1.0534 | test_acc: 0.5733
Epoch: 16 | train_loss: 0.5203 | train_acc: 0.7729 | test_loss: 1.0557 | test_acc: 0.5602
Epoch: 17 | train_loss: 0.4672 | train_acc: 0.8438 | test_loss: 1.5742 | test_acc: 0.5176
Epoch: 18 | train_loss: 0.4261 | train_acc: 0.8208 | test_loss: 1.0783 | test_acc: 0.5653
Epoch: 19 | train_loss: 0.4159 | train_acc: 0.8417 | test_loss: 1.1152 | test_acc: 0.5608
Epoch: 20 | train_loss: 0.3300 | train_acc: 0.8854 | test_loss: 1.2175 | test_acc: 0.5597

```

8. Make a prediction on your own custom image of pizza/steak/sushi (you could even download one from the internet) with your trained model from exercise 7 and share your prediction.

- Does the model you trained in exercise 7 get it right?
- If not, what do you think you could do to improve it?

```

# Get a custom image
custom_image = "pizza_dad.jpeg"
with open("pizza_dad.jpeg", "wb") as f:
    request = requests.get("https://raw.githubusercontent.com/mrdbourke/pytorch-deep-learning/main/images/04-pizza-dad")
    f.write(request.content)

```

```

# Load the image
import torchvision
img = torchvision.io.read_image(custom_image)
img

tensor([[[[154, 173, 181, ..., 21, 18, 14],
         [146, 165, 181, ..., 21, 18, 15],
         [124, 146, 172, ..., 18, 17, 15],
         ...,
         [ 72,  59,  45, ..., 152, 150, 148],
         [ 64,  55,  41, ..., 150, 147, 144],
         [ 64,  60,  46, ..., 149, 146, 143]],
        ...,
        [[171, 190, 193, ..., 22, 19, 15],
         [163, 182, 193, ..., 22, 19, 16],
         [141, 163, 184, ..., 19, 18, 16],
         ...,
         [ 55,  42,  28, ..., 107, 104, 103],
         [ 47,  38,  24, ..., 108, 104, 102],
         [ 47,  43,  29, ..., 107, 104, 101]],
        ...,
        [[119, 138, 147, ..., 17, 14, 10],
         [111, 130, 145, ..., 17, 14, 11],
         [ 87, 111, 136, ..., 14, 13, 11],
         ...,
         [ 35,  22,   8, ...,  52,  52,  48],
         [ 27,  18,   4, ...,  50,  49,  44],
         [ 27,  23,   9, ...,  49,  46,  43]]]], dtype=torch.uint8)

```

```

# View the image
plt.figure(figsize=(10, 7))
plt.imshow(img.permute(1, 2, 0)) # matplotlib likes images in HWC (height, width, color_channels) format not CHW (co
plt.axis(False);

```



```
# Make a prediction on the image
model_4.eval()
with torch.inference_mode():
    # Get image pixels into float + between 0 and 1
    img = img / 255.

    # Resize image to 64x64
    resize = transforms.Resize((64, 64))
    img = resize(img)

    # Turn image in single batch and pass to target device
    batch = img.unsqueeze(0).to(device)

    # Predict on image
    y_pred_logit = model_4(batch)

    # Convert pred logit to pred label
    # pred_label = torch.argmax(torch.softmax(y_pred_logit, dim=1), dim=1)
    pred_label = torch.argmax(y_pred_logit, dim=1) # get same results as above without torch.softmax

# Plot the image and prediction
plt.imshow(img.permute(1, 2, 0))
plt.title(f"Pred label: {class_names[pred_label]}")
plt.axis(False);
```

Pred label: pizza

